

ON BUILDING

SYSTEMS

Multi-Agent Systems and Memory



#ASLI ENGINEERING

BY

ARPIT BHAYANI

Agenda

1. Working memory management + Budgeting 5 mins
2. Memory Architectures
3. Memory Compression and Summarization
4. Memory Write Strategies

10 mins QnA

5. Orchestrator Specialists Pattern
6. Critic Refiner Pattern
7. Mixture of Agents
8. Deadlock

10 mins QnA

Prototypes: 3

System: Incident Auto Remediation System

Appendix:

- Dynamic Agent Spawning
- Blackboard Architecture

Memory Architectures

orchestration ✗ memory ✓

multi-agent system without memory is 'stateless'

every run starts from scratch
specialist re-derives context
that arch already has

1. In-context Memory - current context window

- * requires no retrieval, this is what model attends to

- * ephemeral and bounded

- * compress if it becomes big

extract decisions
relevant facts and discard the transcript.



2. External KV Memory - Redis, DynamoDB holding 'named facts' & decisions

No Vector Retrieval pattern is exact match. "what was rate-limit of this Cx?"
agent must decide if this decision is worth persisting
* you need to know the key, you are asking for!! ElasticSearch

3. Vector Memory - vector databases: pinecone, weaviate, pg-vector

store embedding and retrieval is semantic

- * embedding quality determines retrieval quality

- * top-k contains false positives leading to irrelevant / incorrect results

- * do not consider top-k as ground truth.

pass it through agent and pick relevant ones only.

4. Episodic Memory - seq of events, prior runs, extracted facts

- * compressing episodes into embeddings

loses sequential and causal structured required for complex reasoning

Working Memory Management + Budgeting

#tokens
\$ cost

* always provide
max-steps
max tool calls
max-budged

Context window = agent working memory

↓
limited capacity 1M, 2M, etc (used to be in thousands)

does not mean
every token gets
equal attention!

Challenge: Everything fills up the context window

- ↳ system prompt
- ↳ tool definitions
- ↳ user prompts
- ↳ even responses
- ↳ file contents
- ↳ intermediate observations

So, how to manage context window?

1. Eviction:

sliding window
↑

or
smarter

remove content no longer needed

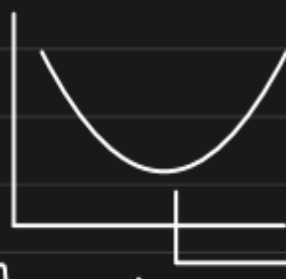
ex: subtask result, reasoning traces, verbose tool outputs

challenge: we don't know if at step 8 we would need step 3 output

Hence summarization is much safer and gen purpose but lossy. at step 15

Use case: coding agent - long running session / refactor

Trading agent - avoid acting on outdated signals



2. Summarization: compress older message & retain summary

this is lossy and involves an LLM call

Use cases: Cx support bot
legal discovery systems
Research assistant

* bit lossy is okay

* having summary is enough

* summary is good enough to capture the 'details'

disk = external

→ coding agents do this

3. Move to external storage: and fetch on demand using semantic search

Use cases: CRM - multi session consistency
Coding agent - multi-session preferences
User preferences over sessions

Prototype: Context window management - 50%, 80% and benchmark

↳ artificial budget = 2000 tokens , 10 step DB migration plan.
Mocking tool outputs

Cooking memory

Memory Compression and Summarization Strategies

1. Rolling Summarization

SUMMARIZE_PROMPT = """

You are summarizing an agent conversation for memory compression.

Preserve:

- All decisions made and the reasoning behind them
- Numerical values, names, and identifiers exactly
- Open questions and unresolved issues
- Any constraints or requirements stated explicitly

Compress:

- Exploratory reasoning that did not lead to a decision
- Repeated restatements of the same point
- Conversational filler

Output format:

DECISIONS: [bullet list]

OPEN_ITEMS: [bullet list]

KEY_FACTS: [bullet list]

NARRATIVE: [1-2 paragraph prose summary]

"""

Every N turns replace actual with summary

Simplest approach, works well

with conversational continuity

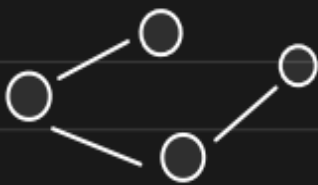
but lossy, factual details might vanish later

(keep decisions separate log)

2. Hierarchical Memory Consolidation

Not taking
Jina Notebook
HML

Extract episodes and keep them updated



preferences change

relationships evolve

Graph is kept updated

Mimics our human brain.

3. Importance-weighted retention

Not all memories are equally worth keeping

Assign importance score

- did this item cause a state change?
- was it explicitly flagged critical?
- has it been accessed multiple times?

→ time, frequency,

contradiction, explicit tagged
as important

let memory decay over time, eventually forgotten.

Personal Assistant user pref decays

You are a memory extraction agent. Your job is to read a set of conversation turns and extract discrete episodic memories worth retaining for long-term agent memory.

For each episode you extract, output a JSON object with this structure:

```
{
  "summary": "One sentence describing what happened or was decided",
  "entities": ["list", "of", "key", "names", "identifiers", "or", "concepts"],
  "decisions": ["Any explicit decisions or conclusions reached"],
  "open_items": ["Unresolved questions or follow-up actions"],
  "importance": 0.0-1.0,
  "tags": ["topical", "tags"],
  "timestamp_hint": "approximate time reference if mentioned, else null"
}
```

Extraction rules:

- Only extract episodes with lasting relevance. Skip small talk, filler, and reasoning that did not lead to a decision or fact.
- One episode per distinct event, decision, or piece of learned context. Do not bundle unrelated things into one episode.
- Importance scoring: 1.0 = critical decision or hard constraint; 0.7 = useful preference or pattern; 0.4 = context that may matter later; below 0.3 = do not extract.
- Preserve names, numbers, and identifiers exactly as stated. Do not paraphrase facts.

Memory Write Strategies

Harder problem: deciding what, when, and how to persist
+ how to ensure it is retrieved reliably when needed

What to persist

not every agent decision is worth storing ↗ irrelevant & contradictory

cost of noisy memory is that retrieval yields ↗ SNR ratio degrades

rule of thumb: persist decisions that would change

the behaviour of agent or future agent execution

persist: policy decisions, user preferences, external observed state,
approvals/rejections with rationale, conflict resolutions.

When to persist

only after decision is confirmed ↗ corruption
partial states are difficult to cleanup

→ finishing subtask, after assembling, critic signing off...

atomicity requirements depends on the usecase

Reliable Retrieval

↗ deterministic & readable enough

1. write canonical keys: user-123 - preference, ...

2. tag memory entries with task-id, agent-id, and timestamp

3. have range lookups available to get past decisions & traces

↳ so that old decisions are not retrieved

Prototype: Evolving Memory User preference

important that you know
you need a multi-agent setup

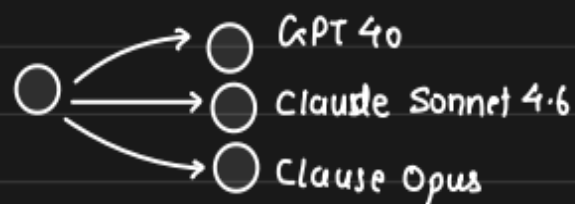
Why multi-agent at all?

a single agent + large context + tools → good enough for most usecase

↳ plus this is simpler as well

1. task can be split
2. subtasks can execute in parallel without shared state

3. you may benefit from
each subtask using diff model



4. token budget exceeds context window

* single agent has high variance
as compared to multi-agent } stronger consistency and predictability

Architectural value is not speed, but determinism and reliability

5. Scoping Responsibility → loosely analogous to microservices

Orchestrator + Specialist

receive, split, dispatch, assemble



specialist agents (cheaper/better)

→ not concatenation, but synthesis

context window overflow
during assembling

- key claims from each
and assemble

→ package subtask with full context

* no clarification should be required by subtask agent

* clear task and response requirements provided

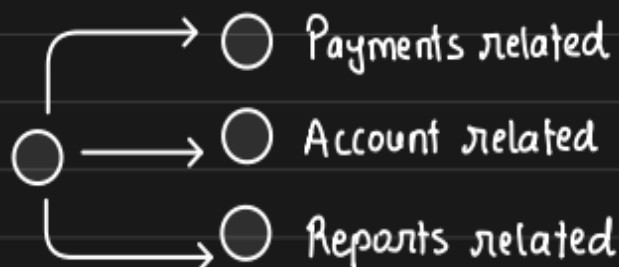
* per task token budget

misclassification
(wrong agent chosen)

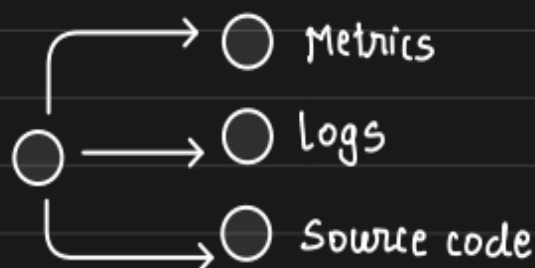
- add classification validation step

Practical use:

Customer Ticket Triage



Incident Triage



Tool Isolation

Prototype : Deep Research Agent

causal, and not parallel

Not all multi-agent exec needs to be parallel

Critic-Refiner

~ second agent whose only job
is to evaluate the output of primary agent
and provides feedback

* correctness $\gg$ speed

↓
has evaluable structure
(tests, quality, constraints, etc)

primary then iterates and improves the output
fixed number of iterations
till some quality threshold
"this is wrong, please try again"

"third function not handling edge case" → actionable signal

should contain: specific location of error,
suggested direction to fix, etc

Practical use:

1. Article moderation : C → unique article R → refines until it meets standards
2. SQL query optimization : C → finds slow queries A → optimizes it
3. legal contract drafting
4. RAG answer quality

task $\neq$ one LLM call

task $\rightarrow$ full agentic loop

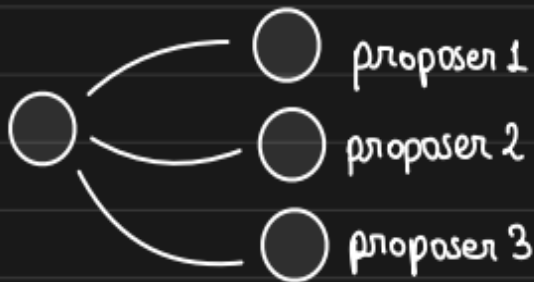
usually outperforms
individual models

Mixture of Agents

exploits "collaborativeness"

$\hookrightarrow$ same task given to multiple specialist agent

aggregator synthesizes all and generates final/best output



1. Time to first token is high

2. Token cost scales linearly w/ proposers

3. proposer diversity matters

Practical use

1. Medical diagnosis - Each agent checks on diff domain (clinical, pathological, etc)
2. News fact checking - each agent checks against diff corpus

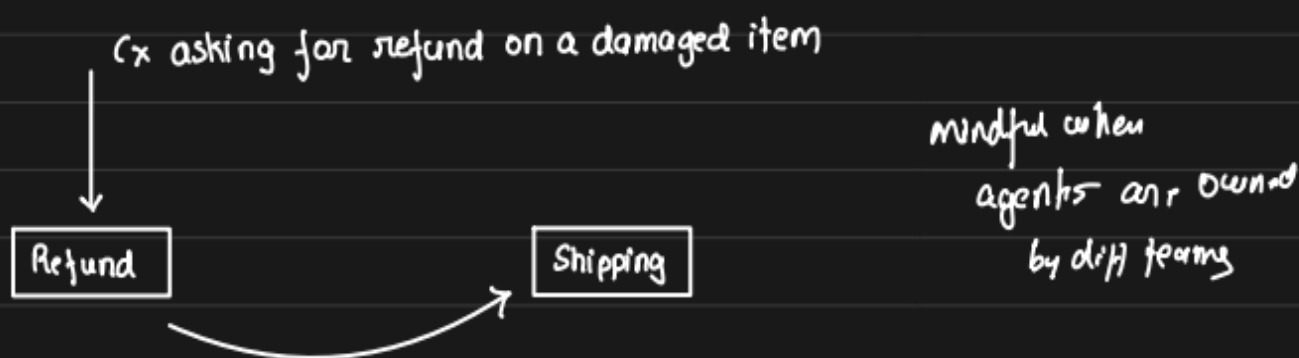
Deadlock in agent pipelines

Agent A produces output that goes to Agent B } deadlock
Agent B produces output that goes to Agent A } diff from traditional deadlocks

a misconfigured handoff rule can bring down the entire pipeline.

↳ we need state machine and cycle detection

1. max iteration count on any loop in the pipeline
2. explicit convergence criteria (most common reason: keeping things open ended)
3. have a supervisor agent to be final authority in declaring the 'result'



"if the item was damaged
escalate to shipping agent
for fault confirmation before processing"

Handoff instruction ↑

"if refund is requested, confirm
with refund agent that refund
is approved before logging
for shipping fault"

Incident Auto-Remediation System

Similar to incident.io

System ingests alerts, triages them, attempts auto remediation,

escalates to on-call if low confidence or triage fails

5000 alerts/day is steady state

Time to First Action $p_{99} = 90\text{sec}$ p_1 , 5min p_2 and 15min p_3

Single Region

Dominant: co-ordination heavy + real-time guarantees + correctness ↑

Solutioning....

1. Functional and Non-functional requirements
2. Key Highlights - Things to keep in mind
3. What is the ceil of # LLM calls per triage?
4. Capacity Estimation - Request volume, storage, compute, LLM
7. should we directly take input from alert engine and act?
5. High level Architecture
 - ↳ Partition key of Kafka? , typical alert behaviour
6. AI 'Pipeline' - list of tools,
 - ↳ what should be output of this 'AI' loop?
7. should we/agent do something after generating output?
8. what if we want to execute as well?

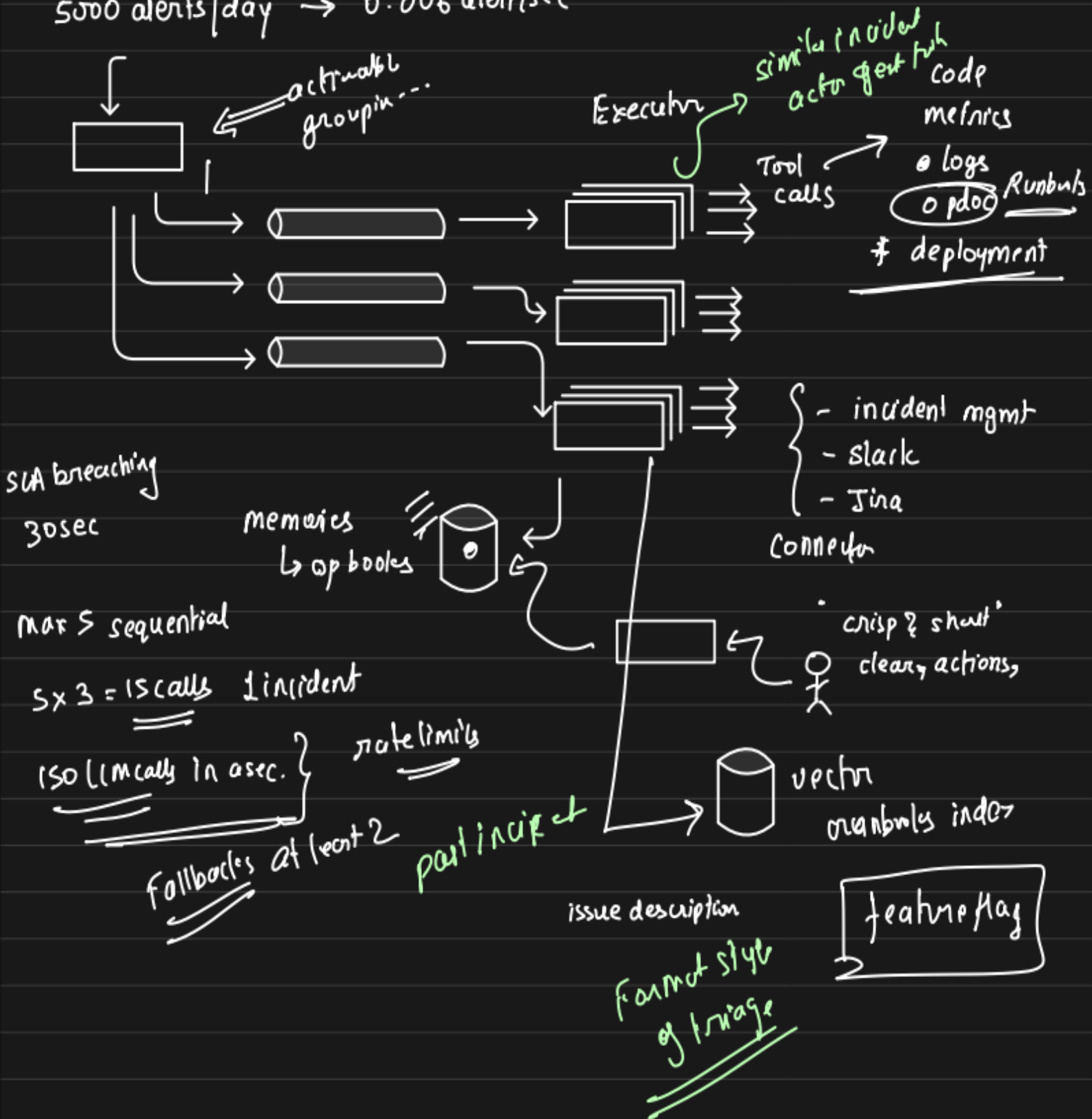
$$5000 \rightarrow \frac{2}{30} \times \frac{10}{3000}$$

1-2 alert/sec

Brainstorm

5000 alerts/day $\rightarrow$ 0.006 alert/sec

'need not act on every one'



augment not replace

Functional Requirements

1. system receives the alert (multiple sources...)
2. what each alert contains - Priority
3. Incident lifecycle
4. Triage
5. Remediation Action
6. Verification and Escalation

Non-Functional Requirements

P1 p99 = 90 sec P2 p99 = 5 mins P3 p99 = 15 min

Availability = 99%

Degradation Policy on LLM?

3-4 serial LLM calls

within 30 seconds

Durability and data loss?

Key Highlights

1. Remediation actions can be irreversible

ex: pod/vm restart, trigger a re-deployment, etc

2. Decision to Action should be fast

3. Conflicting action on same infra via two diff agents / issues

ex: both agents deciding restart of pod

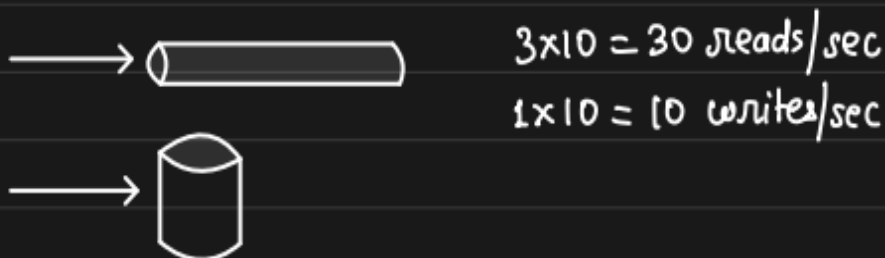
Capacity Estimations

$$5000 \text{ alerts/day} = 0.06 \text{ alerts/sec}$$

$$50\times \text{burst (alerts are cascading)} \rightarrow 3 \text{ alerts/sec}$$

$$3\times \text{headroom} \rightarrow 10 \text{ alerts/sec}$$

For zero data loss, alerts need to be persisted and enqueued



Can LLM become a bottleneck?

Per stage: 4 to 6 LLM calls

10 incidents/sec: 40 to 60 LLM calls

SLA = 90sec



$$90 \times 10 = 900 \text{ incidents}$$

$$\text{total calls} = 4000 \text{ LLM calls}$$



limits per
key...
or

have multiple keys setup

reduced dedupe, low severity, etc

Storage

5000 alerts/day

$$\begin{aligned} \text{Total} &= 100 \times 5000 = 500 \text{ MB/day} \\ &= 15 \text{ GB/month} \end{aligned}$$

per alert storage = 100 KB

Hot/cold, move to S3

timeline, remediation, comment, etc
LLMs resp.

30 day in hot, Rest in cold

SLA = 90sec

1 incident, 1 server = 900 servers peak



$90 \times 10 = 900$ incidents



peak

1 core per incident

because I/O bound

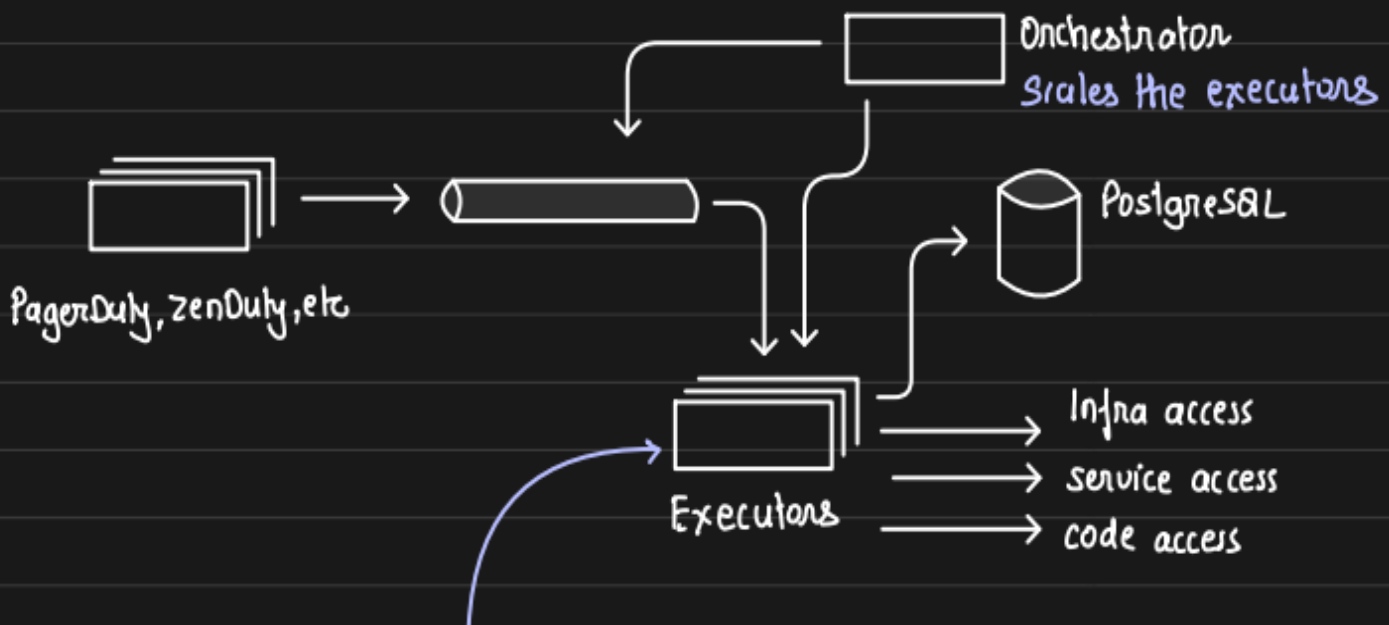
4 core with mem cap, 1 VM $\rightarrow$ 4 incident

Total infra peak = $900/4 = 225$ servers

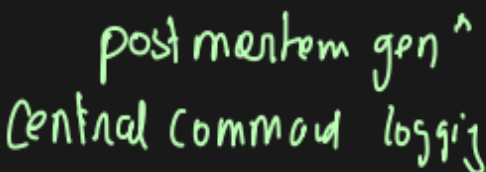
Scaling policy & length of queue

+ most incidents will be duplicates, etc ~ 100 servers peak

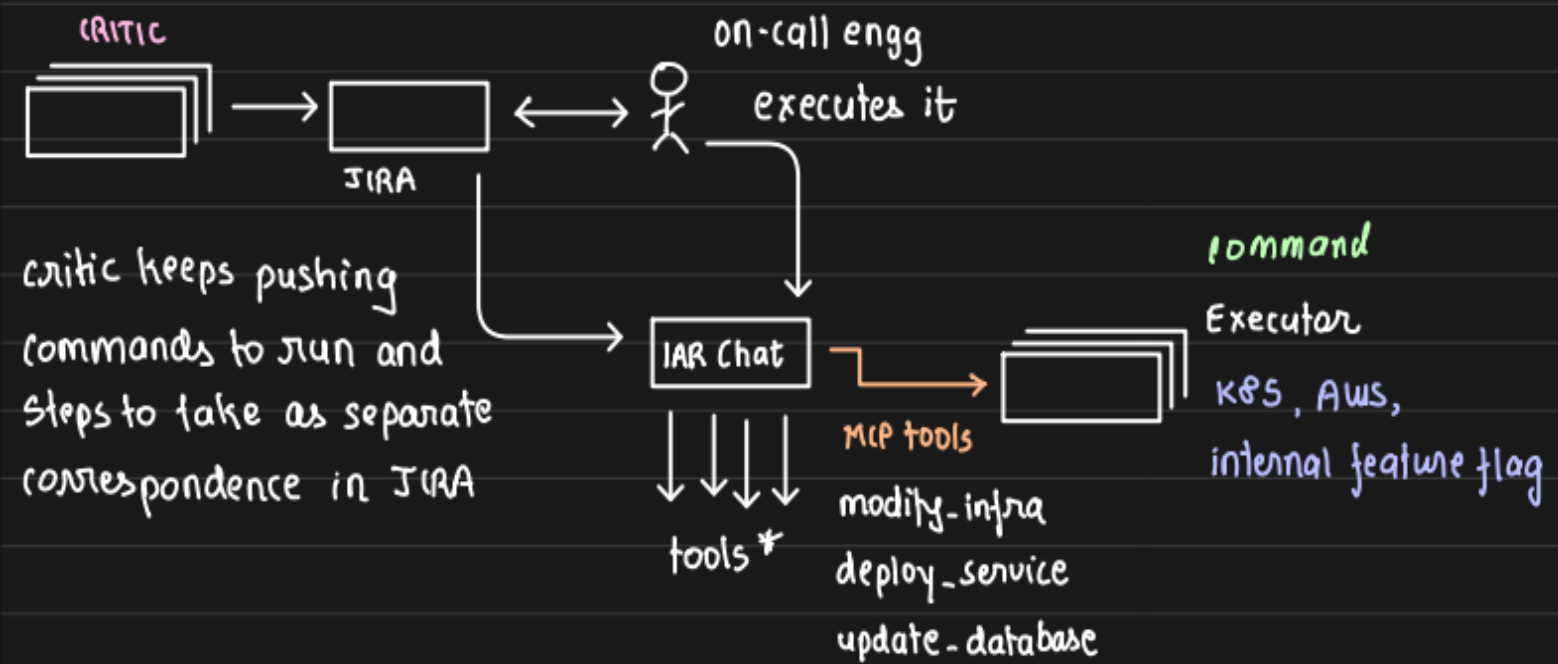
High level Architecture



This is where the
agentic loop runs



if you want AI to even execute....



Post reads and Exercises

You can find them on course portal.

Appendix

Dynamic Agent Spawning

Static Topology - Entry level architecture

fixed set of agents, handoff rules,
graph is known at compile time

Execution path is predictable

Dynamic Spawning

- runtime by observing complexity
- orchestrator detects that a subtask exceeded agent's capabilities
complexity cap, request tool it does not have, explicit signal for help.

AgentSpawn Framework

↳ how much context to transfer

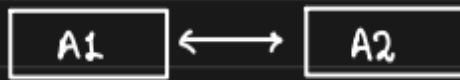
↳ defining capabilities and completion criteria

* Start static → observe degradation → stick to static → still → make dynamic

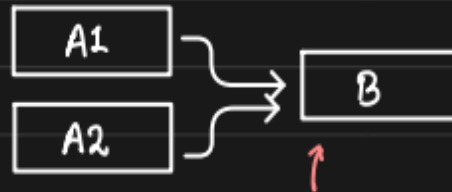
Blackboard Architecture

read ↗ ↘ writes

Oldest co-ordination primitive - shared globally accessible data structure



Exchange messages and each agent maintains its own private state



Agents reads/writes to global state.

Every agent takes entire blackboard as its prompt

expensive ↑

single point of contention

handle conflicting writes

optimizations possible, but blackboard mediates everything

relevant ones are made part of prompt



synchronize or

define a merge behaviour

* Also, what each message on blackboard look like?

- free flowing text
- structured message (fixed schema)